NMAP NETWORK RECONNAISSANCE

Host Discovery, Port Scanning & Service Detection

Network Reconnaissance Lab Report

Subnet Discovery, Localhost Scans and a Controlled SSH Port Test

Kali Linux • Nmap 7.99 • ss • systemctl

Prepared by Anjali · October 2026

1. Executive Summary

This report documents a hands-on network reconnaissance lab performed with **Nmap 7.99** on a Kali Linux virtual machine. The exercises moved from checking the tool, to **host discovery** on the lab subnet, to **port scanning** and service identification, to comparing Nmap's external view of the machine with the local view given by `ss` and `systemctl`.

The report explains host discovery and port scanning in their own sections and shows the commands in practice. Additional exercises cover selected TCP ports, the 20 most common UDP ports, and OS fingerprinting on localhost. Screenshots are cropped to show the specific command and its relevant output. The report records command-usage errors and explains normal scan outcomes. A controlled experiment started SSH, showed TCP port 22 changing from closed to open, and confirmed that SSH was stopped again.

Key Achievements

- Verified the Nmap installation and corrected the "No targets were specified" error
- Identified the lab network (`192.168.186.0/24`) and gateway (`192.168.186.2`) with `ip a` and `ip route`
- Discovered 4 responsive hosts with a ping scan (`-sn`), the host discovery step
- Ran a service/version scan across 256 addresses; found `53/tcp open domain dnsmasq 2.51` on `192.168.186.2`
- Scanned localhost with default ports and with `-p-` (all 65,535 TCP ports reported closed)
- Inspected local sockets with `ss` and compared them with the scan results
- Started SSH in a controlled test and confirmed `22/tcp open ssh OpenSSH 10.0p2 Debian 8` with Nmap
- Stopped SSH and verified `inactive (dead)` with `systemctl status`

2. Introduction

2.1 What is Nmap?

Nmap (Network Mapper) is a free, open-source tool for network discovery and security auditing. It can find responsive hosts, probe TCP and UDP ports, try to identify the service and version behind an open port, and run extra checks through its scripting engine. Nmap **reports observations**: an open port is not proof of a vulnerability, and a version string alone does not show whether a system is patched.

2.2 Why Perform This Exercise?

- Learn the difference between host discovery, port scanning and service/version detection
- Understand the difference between a closed port and a filtered port
- Compare Nmap results with local socket data (`ss`) and service state (`systemctl`)
- Practise capturing reproducible evidence for a cybersecurity portfolio
- Record errors and limitations accurately instead of calling every scan result a vulnerability

2.3 Host Discovery

Host discovery answers the question *"Which addresses on this network are alive?"* It is the first stage of a scan. Checking every port of every possible address would waste time, so Nmap first finds the machines that respond and then scans only those.

How Nmap discovers hosts

Situation	What Nmap sends	Why
Target is on the same local network and Nmap runs with `sudo`	ARP requests ("who has this IP?")	ARP is fast and hard to block, because every host on the segment must answer it. The replies also reveal each host's MAC address.
Target is on a different network	By default: an ICMP echo request (ping), a TCP SYN to port 443, a TCP ACK to port 80 and an ICMP timestamp request	Several different probes are tried because a firewall may block one of them. Any reply marks the host as up.

The ping scan option (-sn)

`-sn` tells Nmap to do host discovery **only** and skip the port scan. The output is simply a list of hosts that answered, with latency and, on a local network, the MAC address. The command is quick and quiet compared with a port scan.

Host discovery in this lab

- `sudo nmap -sn 192.168.186.0/24` checked all 256 addresses of the lab subnet (Figure 5).
- The remote hosts `192.168.186.1`, `.2` and `.254` answered, each with a MAC address beginning `00:50:56` (a VMware virtual-adapter prefix). MAC addresses are normally visible only for hosts on the same network segment, which fits ARP-based discovery.
- The completed service scan later reported **4 hosts up**: the three above plus Kali itself, `192.168.186.133` (Figure 7).

> **WHAT HOST DISCOVERY CAN AND CANNOT TELL YOU**
> It shows that an address answered. It does not show what the machine is, what role it has, or which services it runs. A host that blocks all probes can look "down" even though it is online; the `-Pn` option skips discovery and treats every address as up.

2.4 Port Scanning

Port scanning answers the question *"Which doors are open on this host?"* A computer offers its services through numbered ports (1 to 65535) for TCP and for UDP. Well-known services use well-known ports, for example SSH on 22, DNS on 53 and SMB on 445. A port scan tests ports one by one and records how each one responds.

The TCP SYN scan (-sS), the default with sudo

When Nmap runs with root privileges it uses the SYN scan, shown in the lab as `SYN Stealth Scan` in the progress line (Figure 6). It starts a TCP connection but never finishes it:

Nmap sends	Target replies	Nmap concludes
SYN (a request to connect)	SYN/ACK (agrees to connect)	Open: an application is listening. Nmap then sends a reset (RST) instead of completing the handshake.
SYN	RST (reset)	Closed: the host answered, but no application listens on that port.
SYN	Nothing, or an ICMP "unreachable" error	Filtered: something, often a firewall, is blocking the probe, so the state is unknown.

Without root privileges Nmap uses a **TCP connect scan** (`-sT`) that completes the full handshake. It is slower and more likely to appear in service logs. A **UDP scan** (`-sU`) is a separate technique and was not used in this lab.

Choosing which ports to scan

Option	Ports scanned	Used in this lab
(none)	The 1,000 most commonly used TCP ports	Subnet scan and the first two localhost scans
`-p 22`	Only TCP port 22	SSH test on localhost (Figure 17)
`-p-`	All 65,535 TCP ports	Full localhost scan (Figure 10)

Service and version detection (-sV)

After a port is found open, `-sV` connects to it, sends probes and compares the reply (the "banner") with a database of known services. This is how the lab obtained `dnsmasq 2.51` on port 53 and `OpenSSH 10.0p2 Debian 8` on port 22. The result is what the service reports about itself; it is not proof of the real patch level.

Host discovery and port scanning compared

	Host discovery	Port scanning
Question	Which hosts are alive?	Which ports on a host are open?
Nmap option	`-sn`	Default scan, `-p`, `-p-`, `-sV`
What is probed	Whole addresses	Individual ports of one address
Typical output	List of hosts that are up	Port, state, service, version
Speed	Fast	Slower; grows with the number of ports and silent hosts
Lab example	4 hosts up (Figure 5)	`53/tcp open domain dnsmasq 2.51` (Figure 7)

2.5 Port States

Nmap classifies every scanned port into one of six states. Three of them appeared in this lab.

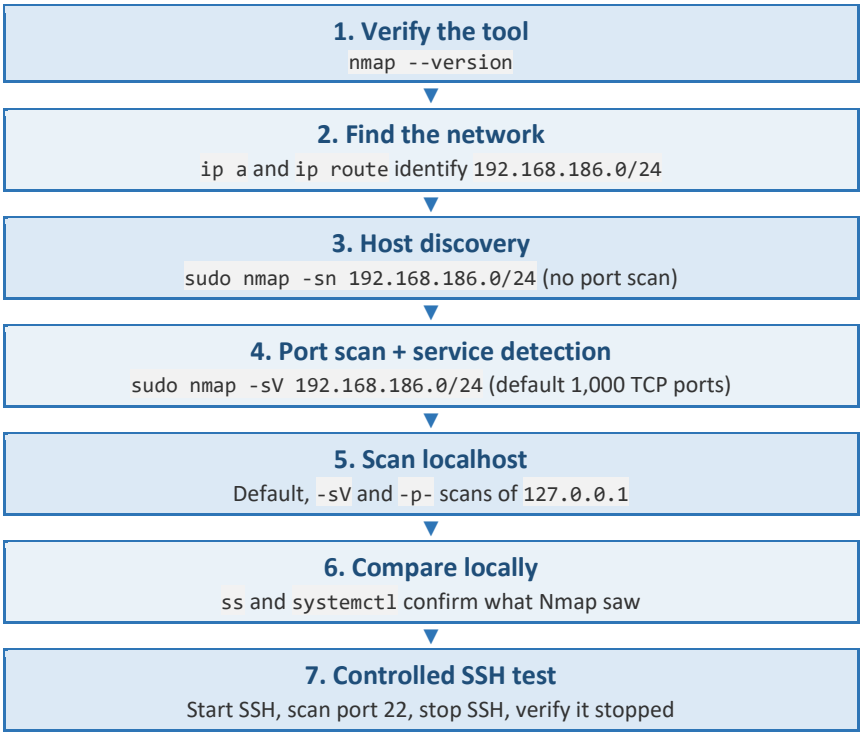
State	Meaning	Seen in this lab?
Open	An application is accepting connections on the port	Yes: port 53 on `192.168.186.2`, port 22 on localhost
Closed	The port answers, but no application is listening (reset received)	Yes: all localhost ports before SSH was started
Filtered	No reply; Nmap cannot tell whether the port is open or closed	Yes: the default ports of `.1`, `.254` and `.133`
Unfiltered	The port answers, but Nmap cannot tell if it is open or closed (ACK scan)	No
Open|filtered	Open or filtered, Nmap cannot decide (common with UDP)	No
Closed|filtered	Closed or filtered, Nmap cannot decide (idle scan)	No

> **AUTHORIZATION**
> Only scan systems and networks you own or have explicit permission to assess. The subnet scan in this report was run against the author's own lab network, and the SSH experiment targeted only localhost.

3. Lab Environment

Item	Observed value
Operating system	Kali Linux (virtual machine)
Nmap version	7.99
Network interface	`eth0`
Kali IPv4 address	`192.168.186.133/24`
Observed subnet	`192.168.186.0/24`
Default gateway (from `ip route`)	`192.168.186.2`
Localhost target	`127.0.0.1`
SSH service	OpenSSH; started for a controlled local port test, then stopped and verified

3.1 Scan Workflow



4. Implementation: Step-by-Step

Each step gives the purpose, the command, the screenshot of that command only, and an explanation of what the output means.

4.1 Step 1: Verifying the Nmap Installation

Before scanning, the tool was checked to confirm it is installed and to record the version, so the results can be reproduced later.

Command used: `nmap --version`

```
┌──(kali㉿kali)-[~]
└─$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.5.2 libssh2-1.11.1 libz-1.3.1 libpcre2-10.48 libpcap-1.10.5 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select
```

Figure 1. *Nmap version check: version 7.99 on x86_64-pc-linux-gnu.*

The terminal showed **Nmap version 7.99** (platform `x86_64-pc-linux-gnu`) together with its build details, so the installation was working.

4.2 Step 2: First Command-Usage Issue (No Target)

The next commands were run with options only:

Command used: `nmap -sn`
`nmap -sV`
`nmap -p-`

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-11 12:42 +0530
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.01 seconds

┌──(kali㉿kali)-[~]
└─$ nmap -sV
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-11 12:43 +0530
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.42 seconds

┌──(kali㉿kali)-[~]
└─$ nmap -p-
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-11 12:43 +0530
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.09 seconds
```

Figure 2. *Three commands without a target: each warns that no targets were specified.*

Each command answered `WARNING: No targets were specified, so 0 hosts scanned.` and finished in a fraction of a second (0.01, 0.42 and 0.09 seconds). The options (`-sn`, `-sV`, `-p-`) describe **how** to scan, but not **what** to scan.

> **ERROR ENCOUNTERED**
> Missing scan target. This was a command-usage mistake, not an installation fault. Cause and fix are in Section 5.1.

4.3 Step 3: Identifying the Network

To choose a correct target, the interface address and routes were read first.

Command used: `ip a`

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c5:43:58 brd ff:ff:ff:ff:ff:ff
    inet 192.168.186.133/24 brd 192.168.186.255 scope global dynamic noprefixroute eth0
       valid_lft 1487sec preferred_lft 1487sec
    inet6 fe80::20c:29ff:fec5:4358/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure 3. *ip a: interface eth0 holds 192.168.186.133/24.*

`ip a` lists interfaces and addresses. `lo` is the loopback interface (`127.0.0.1`); `eth0` is the real adapter, state `UP`, with the IPv4 address `192.168.186.133/24` and broadcast address `192.168.186.255`. `dynamic` and a `valid_lft` lease time show the address was given by DHCP.

Command used: `ip route`

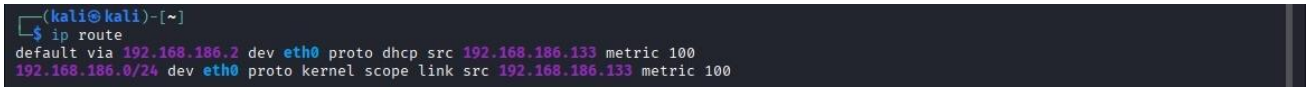


Figure 4. *ip route: the connected subnet and the default gateway.*

`ip route` showed the connected route `192.168.186.0/24` and a default route via `192.168.186.2`. Together, these two commands give the lab subnet that is correct and safe to scan.

4.4 Step 4: Host Discovery (Ping Scan)

This step applies the host discovery technique from Section 2.3: find which addresses answer, without probing any ports.

Command used: `sudo nmap -sn 192.168.186.0/24`



Figure 5. *Ping scan of the lab subnet. The screenshot ends partway through the results.*

- `-sn` means "no port scan": Nmap only checks which hosts are up.
- `sudo` allows Nmap to use ARP on the local network.
- Responses came from `192.168.186.1`, `192.168.186.2` and `192.168.186.254`, each with `Host is up` and a very low latency (about 0.0002 seconds).
- Each remote host reported a MAC address starting `00:50:56`, the VMware virtual-adapter prefix.

> **INTERPRETATION**
> Host discovery shows that addresses respond, not what role they play. The roles of .1, .2 and .254 were not established by this scan alone.

4.5 Step 5: Port Scan and Service Detection of the Subnet

Next, the same subnet was port scanned (Section 2.4) with service detection added.

Command used: `sudo nmap -sV 192.168.186.0/24`

- `sudo` gives Nmap the privileges needed for its default SYN ("stealth") scan.
- `-sV` probes open ports to identify the service and version.
- No `-p` option was used, so Nmap scanned its **default 1,000 most common TCP ports**, not all ports.



Figure 6. *Service scan while still running: the status line shows about 37.5% done.*

Large subnet scans take time. This screenshot was taken mid-scan, so it must not be used for final totals. The line `SYN Stealth Scan Timing: About 37.50% done` confirms that Nmap was using the SYN scan.



Figure 7. *Completed subnet scan: four hosts up, 256 addresses scanned in 217.77 seconds.*

Host	Observed output	Careful interpretation
`192.168.186.1`	1,000 TCP ports filtered (no-response)	No responses were received, so the states are unknown. Filtering is one possible reason.
`192.168.186.2`	`53/tcp open domain dnsmasq 2.51`; 999 closed (reset)	An open DNS-related service was found and Nmap reported dnsmasq 2.51.
`192.168.186.254`	1,000 TCP ports filtered (no-response)	No responses; open or closed could not be determined.
`192.168.186.133`	Host up; 1,000 TCP ports filtered	Kali was up, but the scan received no responses on the default TCP ports.

The summary line was `Nmap done: 256 IP addresses (4 hosts up) scanned in 217.77 seconds`.

4.6 Step 6: Localhost Scans

The loopback address `127.0.0.1` targets the machine itself and gives a clean baseline. Three scans were run, each adding more detail.

Command used: `sudo nmap 127.0.0.1`



Figure 8. *Default scan of localhost: 1,000 TCP ports, all closed, in 0.19 seconds.*

The default scan covers the 1,000 most common TCP ports. All were reported closed (`reset`).

Command used: `sudo nmap -sV 127.0.0.1`



Figure 9. *Service/version scan of localhost: all 1,000 ports closed, no services identified (0.66 s).*

Adding `-sV` found nothing to identify, because service detection only works on open ports.

Command used: `sudo nmap -p- 127.0.0.1`

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -p- 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-11 13:03 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000029s latency).
All 65535 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 65535 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 3.69 seconds
```

Figure 10. *Full TCP port scan of localhost: all 65,535 ports closed (3.69 s).*

`-p-` is shorthand for ports 1 to 65535. The result is not an error: **closed** means Nmap received a reset, so no application was listening on those TCP ports at that moment. UDP, other addresses and other interfaces can differ.

Command	Observation	Duration
`sudo nmap 127.0.0.1`	1,000 TCP ports scanned; all closed	0.19 seconds
`sudo nmap -sV 127.0.0.1`	1,000 TCP ports scanned; all closed; no services identified	0.66 seconds
`sudo nmap -p- 127.0.0.1`	All 65,535 TCP ports closed	3.69 seconds

4.7 Step 7: Inspecting Sockets and the SSH Service State

Nmap shows how a machine looks from the outside. The `ss` and `systemctl` tools show the inside view, so they were used to cross-check the localhost result. In `ss`, `-t` means TCP, `-u` UDP, `-l` listening sockets only, `-n` numeric addresses and ports, `-p` the owning process, and `-a` all sockets.

Command used: `sudo ss -tulnp`

```
┌──(kali㉿kali)-[~]
└─$ sudo ss -tulnp
Netid          State           Recv-Q          Send-Q                    Local Address:Port                  Peer Address:Port          Process
```

Figure 11. *ss -tulnp: only the column headings, so there were no listening sockets.*

Only the column headings were printed: there were no listening TCP or UDP sockets at that moment, which agrees with every port being closed.

Command used: `sudo ss -ant`

```
┌──(kali㉿kali)-[~]
└─$ sudo ss -ant
State              Recv-Q             Send-Q                        Local Address:Port                        Peer Address:Port
SYN-SENT           0                  1                          192.168.186.133:41158                    192.168.186.1:9997
```

Figure 12. *ss -ant: one TCP socket in SYN-SENT state toward 192.168.186.1:9997.*

`ss -ant` lists all TCP sockets, not just listeners. It showed one `SYN-SENT` entry from `192.168.186.133:41158` to `192.168.186.1:9997`. SYN-SENT means a connection attempt was sent and is still waiting for a reply. It is not a listening service.

> **POSSIBLE EXPLANATION FOR THE SYN-SENT ENTRY [confirm]**
> TCP port 9997 is the port Splunk forwarders normally use to send data to a Splunk indexer. Since this lab uses Splunk, this entry may be Kali trying to send logs to `192.168.186.1`, which the subnet scan showed as filtered (no response). This is a hypothesis; the screenshots alone do not prove it.

Command used: `sudo ss -anu`

```
┌──(kali㉿kali)-[~]
└─$ sudo ss -anu
State              Recv-Q             Send-Q                        Local Address:Port                        Peer Address:Port
ESTAB              0                  0                     192.168.186.133%eth0:68                   192.168.186.254:67
```

Figure 13. *ss -anu: a UDP socket between 192.168.186.133:68 and 192.168.186.254:67.*

`ss -anu` showed a UDP socket between `192.168.186.133:68` and `192.168.186.254:67`, ports normally used by DHCP. UDP has no handshake, so the state shown should not be read like a TCP state.

Command used: `systemctl status ssh --no-pager`

```
┌──(kali㉿kali)-[~]
└─$ systemctl status ssh --no-pager
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:sshd(8)
             man:sshd_config(5)
```

Figure 14. *SSH service before the test: loaded, disabled, inactive (dead).*

SSH was loaded but `inactive (dead)` and `disabled`. That explains why nothing listened on port 22. `--no-pager` prints the output directly instead of opening a scrolling viewer.

4.8 Step 8: Controlled SSH Open-Port Experiment

To see a port change from closed to open, the SSH server was started deliberately, checked from the inside, scanned, and then stopped and verified.

Step 8a: Start SSH

Command used: `sudo systemctl start ssh`
`systemctl status ssh --no-pager`

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl start ssh

┌──(kali㉿kali)-[~]
└─$ systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: active (running) since Sun 2026-10-11 13:11:47 IST; 3s ago
 Invocation: 32dee54f00414fbaa6e561ae9462d8aa
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 16962 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 16965 (sshd)
      Tasks: 1 (limit: 4411)
     Memory: 1.9M (peak: 2.4M)
        CPU: 137ms
     CGroup: /system.slice/ssh.service
             └─16965 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 11 13:11:47 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Oct 11 13:11:47 kali sshd[16965]: Server listening on 0.0.0.0 port 22.
Oct 11 13:11:47 kali sshd[16965]: Server listening on :: port 22.
Oct 11 13:11:47 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Figure 15. *SSH started: active (running), with listeners on port 22 in the log.*

The status changed to `active (running)` since 13:11:47. The log reported `Server listening on 0.0.0.0 port 22` and `Server listening on :: port 22`. Those wildcard addresses mean the service listens on all IPv4 and all IPv6 interfaces, subject to network and firewall controls.

Step 8b: Verify the listener locally

Command used: `sudo ss -lntp`

```
┌──(kali㉿kali)-[~]
└─$ sudo ss -lntp
State          Recv-Q         Send-Q                 Local Address:Port                  Peer Address:Port         Process
LISTEN         0              128                          0.0.0.0:22                         0.0.0.0:*             users:(("sshd",pid=16965,fd=6))
LISTEN         0              128                             [::]:22                            [::]:*             users:(("sshd",pid=16965,fd=7))
```

Figure 16. *ss -lntp: sshd listening on 0.0.0.0:22 and [::]:22.*

The output showed TCP listeners at `0.0.0.0:22` and `[::]:22`, owned by `sshd`. This independently confirms the service state without using Nmap.

Step 8c: Scan only port 22

Command used: `sudo nmap -sV -p 22 127.0.0.1`

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -p 22 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-11 13:12 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000094s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.0p2 Debian 8 (protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.62 seconds
```

Figure 17. *Nmap scan of port 22 only: open, ssh, OpenSSH 10.0p2 Debian 8.*

- `-p 22` limits the scan to TCP port 22.
- `-sV` tries to identify the service and version.
- `127.0.0.1` keeps the test on the same machine.
- Result: `22/tcp open ssh OpenSSH 10.0p2 Debian 8 (protocol 2.0)`, scanned in 0.62 seconds.

> **VERSION BANNERS**
> The version string is what the service reported. It does not by itself prove whether all security patches are present.

Step 8d: Stop SSH and verify

The service was stopped so it is not left running unnecessarily, and its state was then checked.

Command used: `sudo systemctl stop ssh`
`systemctl status ssh --no-pager`

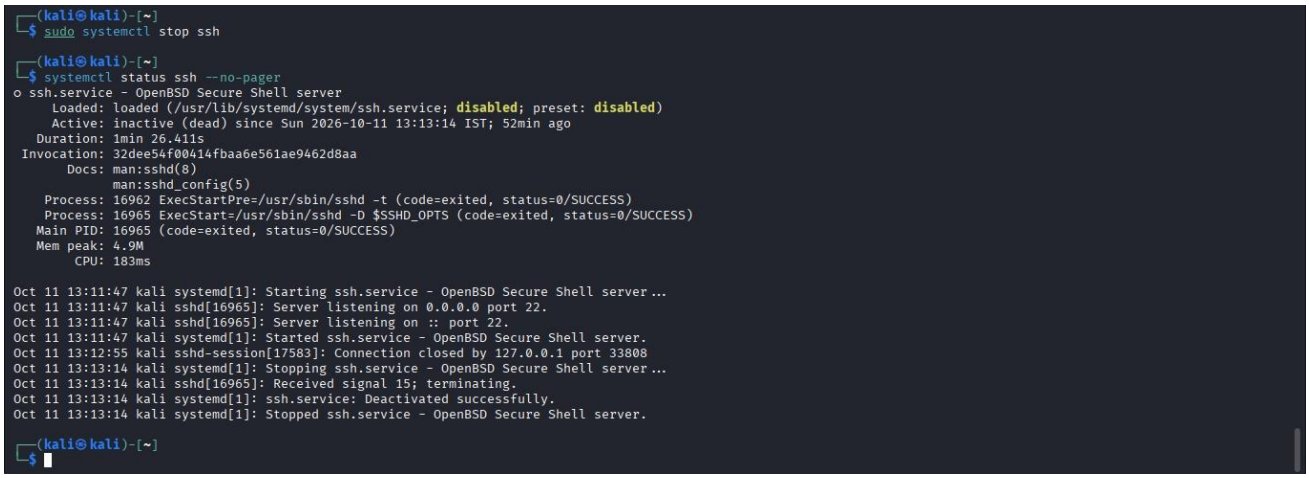


Figure 18. SSH stopped and verified: inactive (dead) since 13:13:14, with the shutdown messages in the log.

What the output shows

- **Active: inactive (dead) since Sun 2026-10-11 13:13:14 IST:** the service is no longer running. The hollow circle (○) before `ssh.service` also marks it as stopped.
- **Duration: 1min 26.411s:** SSH ran for about one and a half minutes, from 13:11:47 to 13:13:14.
- **Main PID: 16965 (code=exited, status=0/SUCCESS):** the SSH process ended normally, with no error.
- **Log lines:** `Stopping ssh.service`, `Received signal 15; terminating.`, `ssh.service: Deactivated successfully.` and `Stopped ssh.service`. Signal 15 (SIGTERM) is the normal "please shut down" request.
- **disabled; preset: disabled:** unchanged. SSH was never set to start at boot, so it will stay off after a reboot.
- **Connection closed by 127.0.0.1 port 33808 at 13:12:55:** a connection from localhost, most likely the probe made by the Nmap service scan that ran at 13:12.

> **VERIFICATION RESULT**
> The "stopped" state is now proven by the status output. The screenshot was taken about 52 minutes after the stop, and the service was still inactive.

5. Challenges & Troubleshooting

This section lists the errors and surprising results from the lab. Most were normal scan outcomes, not faults. Each is written as Problem, Root Cause and Resolution.

5.1 "No targets were specified"

Problem: `nmap -sn`, `nmap -sV` and `nmap -p-` each returned `WARNING: No targets were specified, so 0 hosts scanned.` (Figure 2).

Root Cause: The commands contained options but no target. Nmap needs an IP address, hostname or subnet to know what to scan. This was not an installation problem, since `nmap --version` worked.

Resolution: Add a target after the options:

Command used:
```
sudo nmap -sn 192.168.186.0/24
sudo nmap -sV 192.168.186.0/24
sudo nmap -p- 127.0.0.1
```

5.2 Subnet Scan Showed Filtered Ports

Problem: Hosts `.1`, `.254` and Kali itself showed `1000 filtered tcp ports (no-response)` (Figure 7).

Root Cause: Nmap received no replies, so it could not classify the ports as open or closed. A firewall is one possible reason, but the exact cause is not proven by this evidence.

Resolution: Record the result as "filtered / no response". Do not call it a vulnerability and do not assume the firewall type without further evidence.

5.3 Localhost Scan Showed All Ports Closed

Problem: The default scan and the `-p-` scan of `127.0.0.1` reported every TCP port as closed (Figure 8 to Figure 10).

Root Cause: No TCP application was listening at that time. This is a valid, expected result.

Resolution: Start a controlled local service (SSH, Section 4.8) to compare closed and open behaviour.

5.4 `ss -tulnp` Showed Only Headings

Problem: The listening-socket command printed column titles and no rows (Figure 11).

Root Cause: There were no listening TCP or UDP sockets at that moment.

Resolution: Look at all sockets with `ss -ant` and `ss -anu`, then check service state with `systemctl`.

5.5 SSH Was Inactive (dead)

Problem: `systemctl status ssh` showed `inactive (dead)` and `disabled` (Figure 14).

Root Cause: SSH was installed but not running, and not set to start at boot. That is why port 22 was closed.

Resolution: Start it for the controlled test and verify it, then stop it afterwards if it is not needed:

Command used:
```
sudo systemctl start ssh
sudo ss -lntp
```

5.6 The Subnet Scan Took 217.77 Seconds

Problem: The scan was slow, and a screenshot taken mid-scan showed incomplete results (Figure 6).

Root Cause: Nmap checked 256 addresses and waited for replies from hosts and ports that did not respond (filtered cases take the longest).

Resolution: Let the scan finish before capturing results, and label progress screenshots separately from the completed output (Figure 7).

5.7 No Proof That SSH Had Stopped

Problem: After `sudo systemctl stop ssh` the first screenshot showed only the returned prompt, with no status check, so the stopped state was not proven.

Root Cause: The prompt returning without an error suggests success, but it is not evidence.

Resolution: A status check was run afterwards (Figure 18). It showed `inactive (dead)` since 13:13:14 and a clean exit (`status=0/SUCCESS`), so the issue is resolved:

Command used: `systemctl status ssh --no-pager`

5.8 Summary of Issues

#	Issue	Type	Fix / action
1	No targets specified	Command-usage error	Add an IP, hostname or subnet
2	Filtered ports on subnet hosts	Normal scan result	Record as filtered; do not assume a cause
3	All localhost ports closed	Normal scan result	Start a controlled service to compare
4	`ss -tulnp` showed no rows	Normal observation	Use `ss -ant`, `ss -anu`, `systemctl`
5	SSH inactive (dead)	Service not running	`sudo systemctl start ssh`
6	Scan took 217.77 s	Expected duration	Wait for completion before capturing
7	No proof SSH had stopped	Evidence gap (resolved)	`systemctl status ssh` (Figure 18)

> **NO INSTALLATION ERROR**
> No software-installation error appeared in the screenshots. The only correctable command mistake was the missing target; filtered and closed ports are results, not failures.

6. Verification & Testing

Test	Expected result	Observed evidence	Result
Nmap installed	Version displayed	Nmap 7.99 (Figure 1)	✓ Pass
Target requirement understood	Warning without a target	"No targets were specified" (Figure 2)	✓ Pass
Network identified	Subnet and gateway found	`192.168.186.0/24`, gateway `.2` (Figure 3, Figure 4)	✓ Pass
Host discovery	Responsive hosts listed	`-sn` found `.1`, `.2`, `.254`; 4 hosts up in total (Figure 5, Figure 7)	✓ Pass
Port scan + service detection	Open services identified	`53/tcp open domain dnsmasq 2.51` (Figure 7)	✓ Pass
Localhost baseline	Ports closed	All 65,535 TCP ports closed (Figure 10)	✓ Pass
Local socket check	Matches the scan	No listeners before SSH start (Figure 11)	✓ Pass
SSH listener	Port 22 listening	`0.0.0.0:22` and `[::]:22` (Figure 16)	✓ Pass
Nmap open-port detection	Port 22 open	`22/tcp open ssh OpenSSH 10.0p2` (Figure 17)	✓ Pass

Test	Expected result	Observed evidence	Result
SSH stopped	Service inactive	`inactive (dead)` since 13:13:14 (Figure 18)	✓ Pass

7. Analysis of Findings

7.1 What the Results Show

- **Host discovery:** four hosts were up on `192.168.186.0/24`: `.1`, `.2`, `.254` and Kali (`.133`).
- **Open:** DNS (TCP 53) on `192.168.186.2`, and SSH (TCP 22) on localhost only while it was deliberately started.
- **Closed:** every TCP port on localhost before SSH was started.
- **Filtered:** the default TCP ports of `.1`, `.254` and Kali's own subnet address, where no replies came back.
- **Outside-versus-inside view:** the Nmap results matched `ss` and `systemctl`, which is good evidence the scan was accurate.

7.2 What Should Not Be Concluded

- An open port is **not** a vulnerability by itself.
- A version banner such as `dnsmasq 2.51` shows what the service reports. It is an old release, so it is worth checking in a real assessment, but the banner alone does not prove the system is exposed or unpatched.
- "Filtered" does not identify the cause or the firewall in use.
- "Closed" on localhost says nothing about UDP, other interfaces or ports outside the scan.
- A host found by discovery is not automatically important; its role has to be established separately.

7.3 Defensive Takeaways

- Run only the services you need, and stop test services afterwards (as done with SSH).
- Check listeners regularly with `ss -lntp` and compare them with an external scan.
- Keep services such as dnsmasq and OpenSSH updated and restrict them with a firewall where possible.
- Treat repeated, unexpected connection attempts (for example a stuck `SYN-SENT`) as something to investigate.

8. Command Reference

Command	Why it was used
`nmap --version`	Verify installation and record the version
`ip a`	Find the interface and assigned IP address
`ip route`	Identify the connected subnet and default route
`sudo nmap -sn 192.168.186.0/24`	Host discovery: find responsive hosts without port scanning
`sudo nmap -sV 192.168.186.0/24`	Port scan the default TCP ports and identify services/versions on the subnet
`sudo nmap 127.0.0.1`	Baseline scan of the default TCP ports on localhost

Command	Why it was used
`sudo nmap -sV 127.0.0.1`	Repeat the baseline with service/version detection
`sudo nmap -p- 127.0.0.1`	Scan the full TCP range, 1 to 65535
`sudo ss -tulnp`	List listening TCP/UDP sockets and processes
`sudo ss -ant`	Inspect all TCP sockets, including non-listening states such as SYN-SENT
`sudo ss -anu`	Inspect UDP sockets
`systemctl status ssh --no-pager`	Check SSH service state without opening a pager
`sudo systemctl start ssh`	Start SSH for the controlled port-22 test
`sudo ss -lntp`	Verify TCP listeners and the process bound to each port
`sudo nmap -sV -p 22 127.0.0.1`	Scan only port 22 and identify the service
`sudo systemctl stop ssh`	Stop SSH after the test

9. Limitations & Future Enhancements

9.1 Limitations

- TCP and UDP scans and an OS-detection attempt were performed. No Nmap Scripting Engine (NSE) scan was performed.
- The default subnet scan covered only the 1,000 most common TCP ports.
- The roles of `.1`, `.2` and `.254` were not established.
- No scan output was saved to a file.

9.2 Future Enhancements

- Save results with `-oN` (for example `sudo nmap -sV -oN subnet_scan.txt 192.168.186.0/24`)
- Try selected Nmap Scripting Engine (NSE) checks only inside the authorized lab; use a separate lab VM for more meaningful OS fingerprinting.
- Capture the scan traffic with a packet analyser to see the ARP, SYN, RST and SYN/ACK packets behind host discovery and each port state
- Confirm whether the `SYN-SENT` entry to port 9997 is the Splunk forwarder
- Redact any private details before publishing screenshots or the report on GitHub

9.3 Nmap Installation and Verification

Nmap was already installed in this Kali Linux environment, as confirmed by the successful nmap --version command and the displayed version 7.99. The installation commands below are included so the lab can be reproduced on a fresh Kali installation; the supplied screenshots do not show these installation commands being executed.

sudo apt update

Updates the local package index so Kali can retrieve current package metadata.

sudo apt install nmap -y

Installs Nmap from the configured Kali repositories. The -y option automatically answers yes to package-manager confirmation prompts. Review package changes before using -y on a system where you need manual control.

nmap --version

Verifies that Nmap is installed and prints the version and build information. In this lab, the observed version was Nmap 7.99.

If Nmap is already installed, the installation command does not need to be repeated; the version check is sufficient to document the tool and environment.

9.4 Additional Nmap Exercises Completed

These exercises were performed on 11 October 2026 after the initial subnet and localhost reconnaissance. All targets in this section are the Kali machine's loopback address, 127.0.0.1, keeping the tests local to the lab.

9.4.1 Exercise A — Specific TCP-Port Scan

sudo nmap -p 22,80,443 127.0.0.1

Purpose: checks only TCP ports 22 (SSH), 80 (HTTP) and 443 (HTTPS), rather than Nmap's default set of 1,000 common TCP ports or all 65,535 TCP ports.

Command breakdown: sudo runs Nmap with elevated privileges; nmap launches the scanner; -p 22,80,443 selects the three TCP ports; 127.0.0.1 targets the local Kali system.

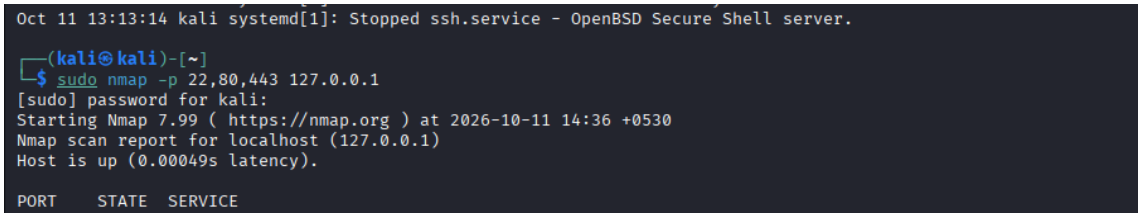


Figure 19. Specific-port scan of localhost: ports 22, 80 and 443 were all reported closed.

Result and interpretation: Nmap reported 22/tcp closed ssh, 80/tcp closed http and 443/tcp closed https. This matched the fact that SSH had been stopped and no HTTP/HTTPS service was listening on these ports at the time of this scan. A closed result means the target responded but no application accepted connections on that TCP port; it is not itself an error or vulnerability.

Cybersecurity relevance: targeted scans are quicker and easier to interpret when an assessment has a defined question, such as checking whether remote-administration or web-service ports are exposed.

9.4.2 Exercise B — Common UDP-Port Scan

sudo nmap -sU --top-ports 20 127.0.0.1

Purpose: checks the 20 most common UDP ports on localhost. UDP is separate from TCP, so a TCP scan alone cannot establish the state of UDP services.

Command breakdown: sudo provides privileges generally useful for UDP scanning; -sU selects UDP scanning; --top-ports 20 selects 20 common UDP ports; 127.0.0.1 is the target.

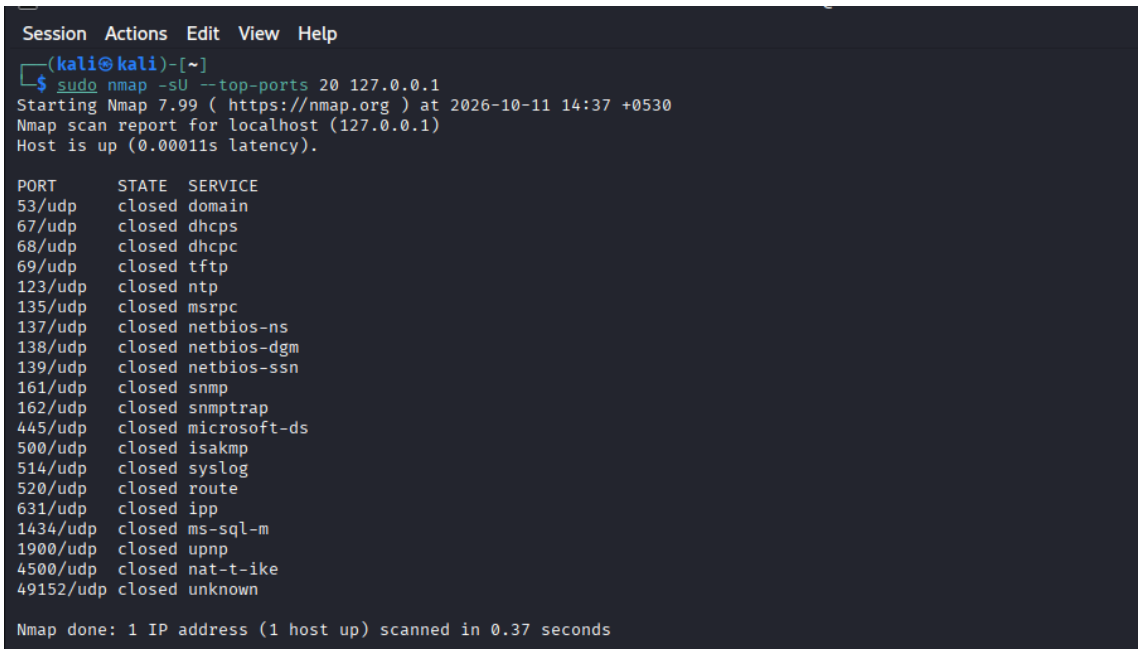


Figure 20. UDP scan of the 20 most common UDP ports on localhost.

Result and interpretation: all 20 listed UDP ports were reported closed, including ports commonly associated with DNS (53), DHCP (67/68), NTP (123) and other services. Nmap received responses indicating these ports were closed at the time of the scan. No open UDP port was reported in this output.

UDP caveat: when a UDP service does not respond, Nmap may report open|filtered because it cannot distinguish an open but silent service from filtering. That state did not appear in this particular output. UDP scans can take longer on other targets because many UDP services do not answer unexpected probes.

Cybersecurity relevance: UDP checks can identify services missed by TCP-only assessments. Analysts use the findings to validate expected services and investigate unnecessary exposure.

9.4.3 Exercise C — OS Detection on Localhost

sudo nmap -O 127.0.0.1

Purpose: asks Nmap to estimate the target operating system from network-stack fingerprinting. The -O option enables OS detection; 127.0.0.1 keeps the experiment local.

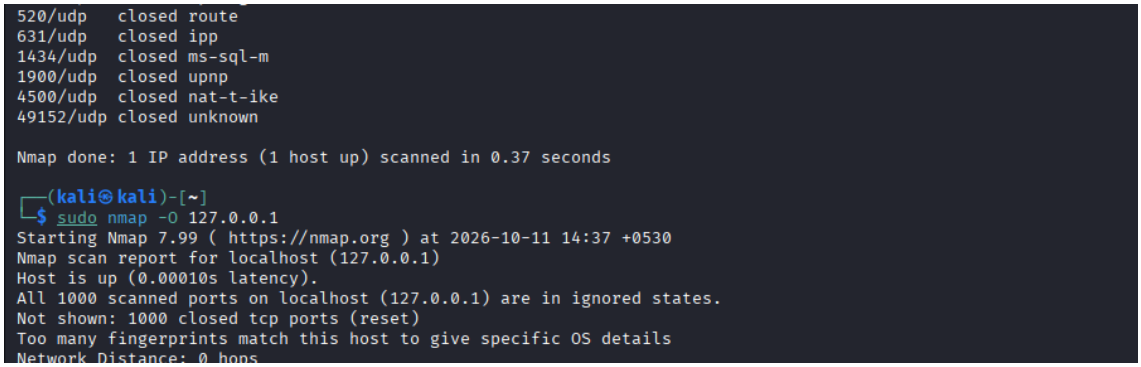


Figure 21. OS-detection attempt on localhost.

Result and interpretation: Nmap reported 'Too many fingerprints match this host to give specific OS details' and 'Network Distance: 0 hops'. This is a limitation of the experiment, not a failed installation. The loopback target is the local host, and the available evidence did not uniquely match one OS fingerprint.

How to improve the experiment: run OS detection against a separate, authorized lab VM with a known IP address, such as a Windows VM on the same isolated virtual network. A separate network stack provides more useful

fingerprint evidence. OS detection is still an estimate and may be affected by firewalls, virtualization and network configuration.

Cybersecurity relevance: OS fingerprinting can support asset inventory and help analysts select appropriate configuration and patch checks, but it should not be treated as definitive proof of an operating system or patch level.

9.5 Results Summary for the New Exercises

Specific TCP ports: 22/tcp, 80/tcp and 443/tcp were closed on localhost. Common UDP ports: all 20 selected ports were reported closed. OS detection: no specific OS details could be determined for localhost because multiple fingerprints matched. These are valid observations from the test conditions, not evidence that the Nmap installation is faulty.

Evidence note: the figures in this added section are cropped to show the command and the corresponding output for that exercise. The existing screenshots and formatting elsewhere in the report have been retained.

10. Conclusion

This lab covered the full basic Nmap workflow: verifying the tool, finding the network, **discovering hosts**, **scanning ports**, detecting services and cross-checking the results with local tools. The most important practical lesson was to **always give Nmap a target** and to read results carefully, because open, closed and filtered mean different things.

The controlled SSH test gave a clear before-and-after example: with the service stopped, port 22 was closed and `ss` showed no listener; after starting it, both `ss` and Nmap showed port 22 open and identified OpenSSH; after stopping it, `systemctl status` confirmed `inactive (dead)`. Every claim in the report is supported by a screenshot of the command that produced it.

> **FINAL OUTCOME**
> A reproducible, well-documented reconnaissance lab that explains host discovery and port scanning, separates scan observations from conclusions, and does not overclaim vulnerabilities.

11. References

1. Nmap Reference Guide: https://nmap.org/book/man.html
2. Nmap, Host Discovery: https://nmap.org/book/man-host-discovery.html
3. Nmap, Port Scanning Basics (port states): https://nmap.org/book/man-port-scanning-basics.html
4. Nmap, Port Scanning Techniques: https://nmap.org/book/man-port-scanning-techniques.html
5. Nmap, Service and Version Detection: https://nmap.org/book/man-version-detection.html
6. `ss` manual page (iproute2): https://man7.org/linux/man-pages/man8/ss.8.html
7. `systemctl` manual page: https://man7.org/linux/man-pages/man1/systemctl.1.html

— End of Report —